Objective 2.3 — Modify scripts for reconnaissance & enumeration

You are tested on **reading and modifying** scripts, not writing them from scratch: identify the language, trace the **input -> processing -> output** pipeline, and make one targeted change. Three languages matter — **Bash**, **Python**, **PowerShell** — shown side by side. Domain 2.0 = 21% of the exam.

1 · Script anatomy

Every recon script has the same six layers. Modifying a script = changing one stage: **input**, **processing**, or **output**.

INPUT	**LIBRARY IMPORTS** `import socket, os, sys`
	CONSTANTS & CONFIG `TIMEOUT = 2 PORTS = [22,80,443]`
PROCESSING	**FUNCTION / CLASS DEFINITIONS** `def check_port() ... class NetworkScanner`
	INPUT VALIDATION (conditionals) `if not sys.argv[1]: usage; exit`
	MAIN LOGIC (loops + conditionals + math) `for host in targets: for port in ports: scan()`
OUTPUT	**OUTPUT / DATA MANIPULATION** `write file · format CSV/JSON · print`

2 · Language ID — three-second tells

Bash `.sh`

- `#!/bin/bash` shebang on line 1
- `echo` for output
- `VAR="val"` assign (**no spaces**) and `$VAR` to reference
- `[ ... ]` tests; `; do ... done`, `fi`
- Calls external tools directly (nmap, curl, grep)

```
#!/bin/bash
TARGET="10.0.0.1"
echo "Scanning $TARGET"
nmap -sS "$TARGET" >> scan.txt 2>&1
```

Python `.py`

- **Indentation** defines blocks (no braces)
- `print()` with parentheses
- `def` functions, `import` libraries, `:` colons
- No `$` on variables
- `sys.argv[1]` for the first argument

```
import socket
TARGET = "10.0.0.1"
for port in [22, 80, 443]:
print(f"Checking {TARGET}:{port}")
```

PowerShell `.ps1`

- **Verb-Noun** cmdlets: `Get-`, `Set-`, `New-`, `Out-`
- `$var` variables; `Write-Host` for output
- `@()` arrays, `@{}` hashtables
- `$_` = current pipeline object
- Text compare operators `-eq -like -match` (not </>)

```
$target = "10.0.0.1"
Get-NetTCPConnection |
Where-Object { $_.State -eq "Listen" } |
Select-Object LocalPort
```

3 · Rosetta table — same operation, three languages

Scan a row to see how each language expresses the same thing.

OP	BASH	PYTHON	POWERSHELL
Comment	`# comment`	`# comment`	`# comment <# multi-line #>`
Assign variable	`VAR="10.0.0.1" # no spaces!`	`var = "10.0.0.1"`	`$var = "10.0.0.1"`
Reference variable	`$VAR ${VAR}`	`var`	`$var`
Print / output	`echo "$VAR"`	`print(var)`	`Write-Host $var`
First CLI argument	`$1`	`sys.argv[1]`	`$args[0] param($t)`
Array / list	`arr=(22 80 443)`	`ports = [22,80,443]`	`$ports = @(22,80,443)`
For over a range	`for i in $(seq 1 254); do ... done`	`for p in ports:`	`for($i=1;$i -le 254;$i++){ }`
Read file line by line	`while IFS= read -r l; do ... done < f.txt`	`for l in open("f.txt"):`	`Get-Content f.txt | ForEach-Object { }`
If / elif / else	`if [ "$p" -eq 80 ]; then ... elif ...; else ...; fi`	`if p==80: ... elif ...: ... else:`	`if($p -eq 80){ } elseif( ){ } else{ }`
Numeric compare	`-eq -ne -lt -gt -le -ge`	`== != < > <= >=`	`-eq -ne -lt -gt -le -ge`
String equals	`[ "$s" = "ssh" ]`	`s == "ssh"`	`$s -eq "ssh"`
Contains / match	`grep -q "Apache" [[ $b == *Apache* ]]`	`"Apache" in b`	`$b -like "*Apache*" $b -match "Apache/\d+"`
AND / OR / NOT	`&& || !`	`and or not`	`-and -or -not`
Arithmetic / increment	`$((a+b)) ((count++))`	`a+b count += 1`	`$a+$b $count++`
Redirect output	`> overwrite >> append 2>&1 errors`	`open(f,"w") / open(f,"a")`	`Out-File Out-File -Append Export-Csv`
Error handling	`if [ $? -ne 0 ]; then ...`	`try: ... except: ...`	`try { } catch { }`
Define function	`f(){ local x=$1; ... }`	`def f(x): ...`	`function F { param($x) ... }`
Load capability	`(calls tools: nmap, curl, grep)`	`import socket`	`Import-Module PowerView`

4 · Concept cards

The 2.3 concepts grouped by role, each ending in the exam tell.

WHAT SCRIPTS DO (THE PIPELINE)

Information gathering

Automate collecting target data — the script is a pipeline: **input -> processing -> output**.

- Input = targets/ports/wordlists/args; Processing = loops+tools; Output = screen/files
- Modifying a script = change **one stage**: what it targets, what it does, or where results go
- Passive scripts query third-party sources; active scripts probe the target directly

EXAM TELL "target a different subnet" = change the IP-range variable or loop bounds; > overwrites, >> appends.

Data manipulation

Turn raw tool output into actionable intel: **parse, filter, sort, format**.

- Bash: `grep` `cut` `awk` `sed`; Python: `split()` + regex; PowerShell: `Where-Object` / `Select-Object`
- Structures: string, list/array, dict/hashtable, CSV row, JSON object
- Format to CSV/JSON/HTML for reporting; handle missing/malformed data gracefully

EXAM TELL "extract specific fields" = parsing; "convert to CSV" = output formatting for the report.

LOGIC CONSTRUCTS

Loops

Repeat a block — the automation engine that scales across many targets.

- **for** = known collection (a `/24` sweep is 1-254, a port list, lines of a file)
- **while** = unknown length (retries, polling, reading a stream)
- **do** = runs at least once before checking; `break` exits, `continue` skips to next

EXAM TELL fixed collection -> for; unknown duration -> while; must run once -> do.

Conditionals

Make decisions from comparisons/tests — what makes a script adaptive.

- `if [ -z "$1" ]` at the top = **input validation** (most common use)
- Result branching runs extra actions only when a condition is met
- Exit-code checks: `$?` (Bash), `$LASTEXITCODE` (PowerShell)

EXAM TELL `if [ -z "$1" ]` -> input validation; check `$?` after a tool to detect failures.

Boolean operators

Combine conditions into one expression.

- AND: `&&` (Bash) / `and` (Py) / `-and` (PS)
- OR: `||` / `or` / `-or`
- NOT: `!` / `not` / `-not`
- Patterns: alive **AND** in-scope; port 80 **OR** 8080; **NOT** in exclusion list

EXAM TELL know the three per language: `&&/||/!`, `and/or/not`, `-and/-or/-not`.

String operators

Compare, search, and slice text — banners, hostnames, versions are all strings.

- Python: `in` (substring), `startswith()`, slicing `b[0:6]` (zero-based)
- PowerShell: `-like` (wildcard) vs `-match` (regex); `-ceq` case-sensitive
- Bash: string equality is `=` **inside** `[ ]` (not `==`)

EXAM TELL `-like` = wildcard, `-match` = regex; banner parsing is the classic use.

Arithmetic operators

Do math — counters, ranges, thresholds, progress.

- Bash **requires** `$(( ))` — `$count + 1` concatenates strings (trap!)
- Increment: `((count++))` / `count += 1` / `$count++`
- Modulo `%` for progress reporting; thresholds like `>= 7.0` for CVSS priority

EXAM TELL Bash needs `$(( ))` for real math; `%` shows up in progress-report questions.

STRUCTURE: LIBRARIES / FUNCTIONS / CLASSES

Libraries

Pre-built capability you import instead of writing from scratch.

- Python: `socket` (net), `requests` (HTTP), `scapy` (packets), `re` (regex), `paramiko` (SSH)
- PowerShell: `Import-Module` (PowerView / ActiveDirectory / PowerSploit)
- Bash has no imports — external tools (nmap/curl/grep) are its "libraries"

EXAM TELL the `import` line tells you the capability: `socket`=networking, `scapy`=packet crafting.

Functions

Named, reusable blocks: take inputs, do a task, optionally return.

- `def f(x, timeout=2):` (Py) / `function F {param($x)}` (PS) / `f(){ local x=$1; }` (Bash)
- Default parameters give fallback values; `return` sends output back
- Name describes purpose (`grab_banner`); modify one function to change one behavior

EXAM TELL read the name for intent, the params for inputs, the `return` for output.

Classes

Templates bundling related data + methods into objects.

- `class`, `__init__` = constructor, `self` = the current instance, instance vars store per-object data
- Instantiate: `s = NetworkScanner("10.0.0");` call methods `s.discover_hosts()`
- On the exam you read/modify a class, not write one from scratch

EXAM TELL `self` + `__init__` = a class; you tweak method logic, add a param, or change stored data.

5 · Fix the script

The planted bugs the exam uses — buggy vs fixed, with the reason.

Bash Spaces around = break assignment

BUGGY

```
TARGET = "10.0.0.1"
nmap -sS "$TARGET"
```

FIXED

```
TARGET="10.0.0.1"
nmap -sS "$TARGET"
```

WHY With spaces, Bash reads TARGET as a command and "=" as an argument. Assignment must have no spaces around the equals sign.

Bash Overwrite vs append in a loop

BUGGY

```
while IFS= read -r t; do
nmap "$t" > scan.txt
done < targets.txt
```

FIXED

```
while IFS= read -r t; do
nmap "$t" >> scan.txt
done < targets.txt
```

WHY "> " truncates the file every iteration, so only the last host survives. ">>" appends each host's results.

Bash stderr redirection order

BUGGY

```
nmap "$t" 2>&1 > scan.txt
```

FIXED

```
nmap "$t" > scan.txt 2>&1
```

WHY Redirections apply left-to-right. In the buggy form 2>&1 copies stderr to the OLD stdout (terminal) before stdout is sent to the file, so errors miss scan.txt.

Bash Arithmetic needs $(( ))

BUGGY

```
count=0
count=$count + 1 # want to add
```

FIXED

```
count=0
count=$((count + 1)) # or ((count++))
```

WHY Outside $(( )), Bash treats values as strings, so you get the literal "0 + 1", not 2. Use $(( )) or (( )) for math.

Bash > is redirection inside [ ]

BUGGY

```
if [ "$count" > 5 ]; then ...
```

FIXED

```
if [ "$count" -gt 5 ]; then ...
```

WHY Inside single [ ], ">" is output redirection (it even creates a file named 5). Numeric comparison uses -gt / -lt / -eq.

Python Indentation defines the block

BUGGY

```
for port in ports:
print(port) # not indented
```

FIXED

```
for port in ports:
    print(port) # 4-space indent
```

WHY Python uses indentation, not braces, to mark a block. A missing/misaligned indent raises IndentationError.

Python Cast argument to int

BUGGY

```
port = sys.argv[1]
sock.connect((host, port))
```

FIXED

```
port = int(sys.argv[1])
sock.connect((host, port))
```

WHY Command-line args arrive as strings. A socket needs an integer port, so cast with int() before connecting.

PowerShell Use -gt, not > , to compare

BUGGY

```
if ($port -eq 80 -and $count > 3) { }
```

FIXED

```
if ($port -eq 80 -and $count -gt 3) { }
```

WHY In PowerShell ">" is redirection. Numeric comparison uses -gt (and -lt/-eq/-ne), because <,> are reserved for redirection.

6 · Read-the-code quiz (answer key included)

Identify the language, say what a snippet does, choose the modification, or predict output.

Q01 · Identify the language

```
#!/bin/bash
for h in $(cat hosts.txt); do echo "$h"; done
```

Which language?

Bash Bash

WHY Shebang #!/bin/bash, echo, $h with no-space usage, and $(...) command substitution are all Bash tells.

Q02 · Identify the language

```
import socket
def scan(host, port):
print(f"{host}:{port}")
```

Which language?

Python Python

WHY Indentation-defined block, def, import, print() with parentheses, f-string — unmistakably Python.

Q03 · Identify the language

```
Get-NetTCPConnection |
Where-Object { $_.State -eq "Listen" } |
Select-Object LocalPort
```

Which language?

PowerShell PowerShell

WHY Verb-Noun cmdlets, the pipeline, $_ (current object), and -eq operator are PowerShell.

Q04 · What does it do?

```
for i in $(seq 1 254); do
ping -c 1 192.168.1.$i
done
```

What does this do?

/24 ping sweep (host discovery) Bash

WHY It iterates host octets 1-254 and pings each — a classic Class-C host-discovery sweep.

Q05 · Modify it

```
RANGE="192.168.1"
for i in $(seq 1 254); do nmap "$RANGE.$i"; done
```

Modify it to scan 10.0.0.0/24 — what changes?

Change RANGE="10.0.0" Bash

WHY The target is defined once in the RANGE variable. Changing input is the whole modification — leave the loop alone.

Q06 · Predict output

```
ports = [22, 80, 443]
print(ports[-1])
```

Predict the output.

443 Python

WHY Negative indexing counts from the end, so ports[-1] is the last element, 443.

Q07 · Spot the bug

```
TARGET = "10.0.0.1"
nmap -sS "$TARGET"
```

Spot the bug (Bash).

Remove the spaces: TARGET="10.0.0.1" Bash

WHY Spaces around = break Bash assignment; it tries to run TARGET as a command.

Q08 · Modify it

```
nmap -sS "$t" > out.txt
```

You must keep results from every host in a loop. What do you change?

> becomes >> Bash

WHY > overwrites each iteration; >> appends, preserving all hosts.

Q09 · What capability?

```
from scapy.all import *
```

What capability does this add?

Packet crafting / analysis Python

WHY scapy is the packet-crafting library; import socket would be plain networking, requests would be HTTP.

Q10 · Modify it

```
nmap -sS "$target"
```

The scenario says "reduce detection risk." What kind of change fits?

Add timing delays / lower aggressiveness (e.g., -T1, sleep in loop) Bash

WHY "Reduce detection" maps to slowing the scan (timing template, added sleeps), not changing scope or output.

Q11 · What does it do?

```
try:
s.connect((host, port))
except socket.timeout:
print("timed out")
```

What is the try/except doing here?

Preventing a crash on unreachable hosts Python

WHY Network probes fail often; try/except catches the timeout so the loop continues instead of aborting.

Q12 · Predict output

```
Get-Process |
Where-Object { $_.CPU -gt 10 } |
Select-Object Name
```

What does $_ mean, and what is returned?

$_ = current pipeline object; returns names of processes using >10 CPU PowerShell

WHY Where-Object filters each piped object via $_, then Select-Object Name projects just the Name property.

7 · Keyword → meaning reference

KEYWORD / SYNTAX	MEANING	LANG
#!/bin/bash	Identifies a Bash script	Bash
echo	Bash output to screen	Bash
$1 $2	Bash command-line arguments	Bash
> >>	Redirect: overwrite / append	Bash
2>&1	Capture stderr with stdout	Bash
$?	Exit code of last command	Bash
$(( ))	Bash arithmetic expansion	Bash
[-z "$1"]	Empty-string / input check	Bash
seq 1 254	Number sequence for a /24 loop	Bash
IFS= read -r	Read a file line by line	Bash
-eq -ne -lt -gt	Bash/PowerShell numeric compare	All
= (in [ ])	Bash string equality	Bash
import	Load a Python library	Python
def	Define a Python function	Python
print()	Python output	Python
sys.argv[1]	Python first argument	Python
try / except	Python error handling	Python
int() str()	Python type casting	Python
with open()	Python file read/write	Python
in	Python substring / membership	Python
indentation	Python block definition (critical)	Python
self	Current class instance	Python
__init__	Python class constructor	Python
class	Object template	Python
Get- / Set- / New-	PowerShell Verb-Noun cmdlet	PowerShell
$variable	PowerShell variable	PowerShell
Write-Host	PowerShell output	PowerShell

KEYWORD / SYNTAX	MEANING	LANG
`$_`	Current pipeline object	PowerShell
`Where-Object`	Filter pipeline objects	PowerShell
`Select-Object`	Choose properties	PowerShell
`Out-File -Append`	Write / append to file	PowerShell
`[PSCustomObject]`	Structured output	PowerShell
`-like -match`	Wildcard / regex match	PowerShell
`Import-Module`	Load a PowerShell module	PowerShell
`for loop`	Fixed iteration (known collection)	All
`while loop`	Conditional repetition	All
`do loop`	Runs at least once	All
`break continue`	Exit loop / skip iteration	All
`and / or / not`	Boolean logic (per-language syntax)	All
`modulo %`	Progress / remainder	All
`input -> processing -> output`	Script pipeline model	All